The AI Execution Layer: A Minimal Runtime Architecture for AI-Native Systems

Wang Luqiang
ai-lib

March 2026

Preprint Version: 1.8

Abstract

Rapid growth in AI capabilities has produced a fragmented integration landscape in which applications must connect to heterogeneous providers, tools, services, and devices through incompatible APIs. This paper introduces the AI Execution Layer, a minimal runtime architecture that standardizes capability execution while keeping orchestration logic outside the runtime boundary.

The system is formalized as a three-layer logical model, $S = (A, P, E)$, where A denotes cognition, P denotes the Contact Layer (routing and policy), and E denotes execution. This logical model is mapped to the broader five-layer AI-Native implementation stack to separate architectural invariants from deployment structure.

Capabilities are unified as $C = (I, O, \Sigma, Q, P)$, where I and O are schemas, Σ specifies execution semantics, Q specifies quality telemetry, and P specifies pricing metadata. The execution runtime collects and transports Q and pricing metadata without making policy decisions.

Minimality is enforced as a design constraint: the runtime performs deterministic execution functions only, including protocol translation, capability invocation, result normalization, connection management, bounded retry semantics for transient failures, error propagation, and session context forwarding. By excluding planning, workflow orchestration, and provider selection, the runtime remains lightweight, stateless, and horizontally scalable while supporting streaming, polling-based asynchronous tasks, input validation, and baseline security controls.

Keywords: AI-Native Architecture, Execution Layer, Capability Abstraction, Runtime Systems, AI Infrastructure

1 Introduction

1.1 The Integration Challenge

AI applications increasingly depend on heterogeneous capabilities provided by model vendors, external services, and local devices. Providers such as OpenAI [1], Anthropic [2], Google, and DeepSeek expose powerful models through APIs, but direct integration introduces tight coupling and escalating system complexity.

Most current AI frameworks address this challenge through application-level orchestration (e.g., LangChain [3]). Although this approach improves development velocity, it frequently combines orchestration policy and execution mechanics in one runtime, thereby increasing lock-in and reducing component reusability.

1.2 The Missing Layer

The ecosystem still lacks a minimal execution layer dedicated to capability invocation. Such a layer should:

1. Execute capabilities defined by standardized protocols
2. Translate protocol requests into provider-specific API calls
3. Normalize results into standardized responses
4. Handle execution errors without implementing application-level retry strategies

This layer excludes higher-level concerns such as planning, workflow orchestration, and application logic, thereby preserving a lightweight and reusable execution boundary.

1.3 Contributions

The primary contributions are as follows:

1. **Formal Architectural Model:** A three-layer logical model $S = (A, P, E)$ is specified to separate cognition, orchestration, and execution with explicit layer responsibilities.
2. **Minimal Runtime Constraint:** Minimality is formalized as a verifiable design constraint, with explicit runtime responsibilities and non-responsibilities for deterministic capability execution.
3. **Unified Capability Semantics:** Capability is formalized as $C = (I, O, \Sigma, Q, P)$, enabling technical invocation and economic observability to share a single schema-level object.
4. **Cross-Layer Integration:** The three-layer logical model is mapped to the five-layer AI-Native implementation stack, clarifying how execution-layer invariants are preserved in full-system deployments.

The execution layer is then positioned within the broader five-layer AI-Native Architecture developed in later sections.

2 Formal Architectural Model and Problem Formulation

2.1 Logical Model: $S = (A, P, E)$

The AI execution architecture is defined as the following logical tuple:

$$S = (A, P, E)$$

where:

- A represents the **Agent Intelligence Layer** (reasoning and planning),
- P represents the **Contact Layer** (routing and decision-making),
- E represents the **Execution Layer** (capability invocation).

The model separates the AI system into three distinct layers, each with clearly defined responsibilities:

Agent Layer: Agents represent intelligent behavior. They reason about tasks, plan actions, and decide which capabilities should be invoked. This layer handles:

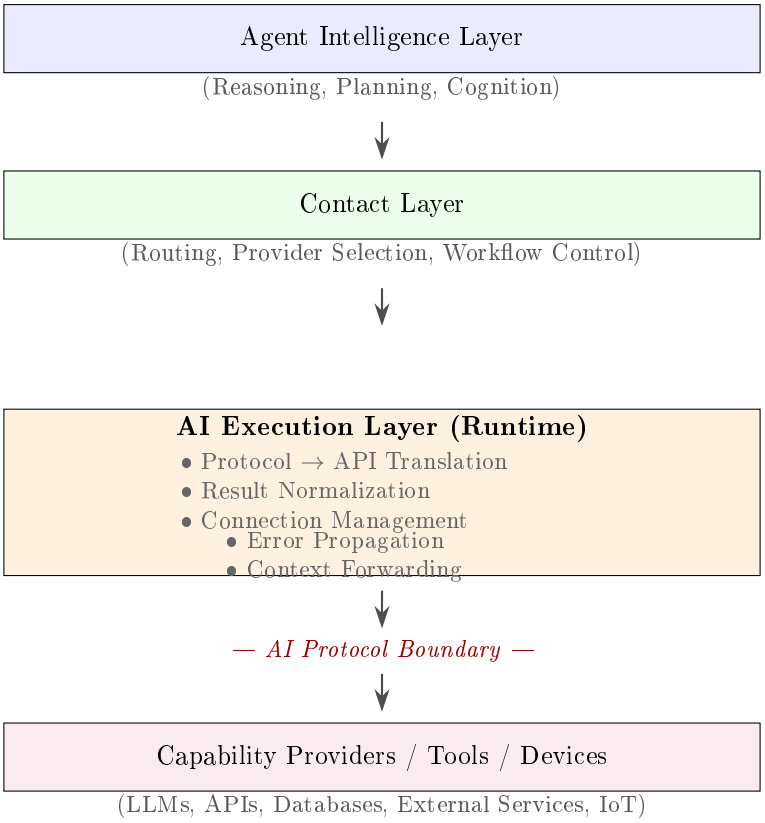

Figure 1: Layered architecture of the AI Execution Layer within AI-native systems, illustrating the separation between cognition, the Contact Layer, and capability execution.

- Understanding user goals
- Planning sequences of actions
- Making decisions about capability requirements
- Managing conversation context and memory

Contact Layer (P): The Contact Layer determines *how* capabilities are selected and composed. This layer is responsible for:

- Capability selection based on agent intent
- Provider preference and selection
- Routing strategies
- Cost or latency optimization
- Policy enforcement

Execution Layer: The execution layer is responsible *only* for executing capability calls. It does not implement routing policies, provider selection, or agent logic. This design keeps the runtime minimal and reusable.

2.2 Logical Model vs. Implementation Stack

In this paper, $S = (A, P, E)$ is the **logical model**. The five-layer AI Native Stack presented later is an **implementation stack** used to realize this model in production systems.

- The logical model specifies invariants and boundaries.
- The implementation stack specifies deployment surfaces and engineering composition.

This distinction avoids a common modeling conflict: a system can be five-layer in implementation while remaining three-layer in execution logic.

2.3 Rationale for Separation

This separation addresses a critical architectural problem observed in existing frameworks. Many systems conflate orchestration responsibilities with execution mechanisms, resulting in:

- **Framework Lock-in:** Applications become tightly coupled to specific orchestration frameworks
- **Limited Reusability:** Runtime components cannot be easily reused across different applications
- **Complexity Creep:** Execution runtimes grow to include policy logic, making them difficult to maintain

By separating these concerns, we enable:

- **Runtime Portability:** Execution runtimes can be reused across different orchestration strategies
- **Policy Flexibility:** Organizations can implement custom routing policies without modifying the runtime
- **Clear Boundaries:** Each component has well-defined inputs, outputs, and responsibilities

3 Why Minimal Execution Matters

Minimality is treated as a system constraint rather than a stylistic preference. Embedding policy logic into the execution runtime introduces state coupling, reduces portability, and increases cross-layer failure propagation. Minimality is therefore defined as follows:

> **Minimality Constraint:** The execution layer MUST implement deterministic capability invocation only, and MUST NOT perform policy-driven decisions such as provider selection, strategic retry, or workflow-level fallback.

Under this constraint, runtime semantics are specified through core responsibilities and explicit non-responsibilities.

3.1 Core Responsibilities

The runtime serves as a standardized capability execution engine. Its responsibilities are intentionally limited to six core functions:

1. Capability Invocation: Execute a capability defined in the AI Protocol. The runtime receives standardized capability requests and invokes the appropriate execution path.

2. Protocol Translation: Convert protocol-level requests into provider-specific API calls. This translation layer enables applications to work with standardized capability definitions while the runtime handles provider-specific implementations.

3. Result Normalization: Return standardized responses to the caller. Regardless of which provider executed the capability, the runtime normalizes results into a consistent format.

4. Connection Handling: Manage network communication with providers. This includes connection pooling, timeout management, and network-level error handling.

5. Error Propagation: Report execution errors without implementing application-level retry strategies. The runtime distinguishes between execution errors (which it reports) and retry logic (which belongs in the orchestration layer).

5.1. Bounded Retry Semantics (Optional): For transient network failures (e.g., connection timeouts, 429 rate limits), the runtime MAY implement limited micro-retries—typically 1–2 immediate retries with exponential backoff at the network layer. This differs from application-level retry strategies in that:

- Micro-retries are bounded and automatic
- They address only transient network-level failures
- They do not implement policy-driven retry logic (e.g., provider fallback)
- Total retry duration must remain within the execution timeout budget

This choice reflects production constraints: transient network failures are common, and surfacing every transient event to the orchestration layer introduces avoidable latency and cross-layer communication overhead.

6. Session Transport: Forward session context without interpreting application policy. The runtime transports contextual information required for capability execution but does not interpret or modify policy decisions.

3.2 What the Runtime Does NOT Do

To maintain its minimal design, the runtime explicitly avoids:

- **Agent Planning:** Deciding what sequence of capabilities to invoke
- **Workflow Orchestration:** Managing complex multi-capability workflows
- **Application Logic:** Implementing business logic specific to applications
- **Provider Selection:** Choosing which provider should execute a capability
- **Routing Policies:** Implementing cost optimization or latency-based routing
- **Application-Level Retry Strategies:** Determining when and how to retry failed executions across providers or workflows

These responsibilities belong in the orchestration and agent layers, maintaining clear separation of concerns.

3.3 Stateless Execution Design

The runtime operates as a stateless execution engine. Each capability invocation is independent, with any necessary state maintained by the calling agent or orchestration layer. This design enables:

- **Horizontal Scalability:** Multiple runtime instances can be deployed without state synchronization
- **Simplified Debugging:** Each execution is self-contained and traceable
- **Fault Isolation:** Failures in one execution do not affect others

3.4 Asynchronous and Long-Running Tasks

Modern AI capabilities extend beyond synchronous text generation to include long-running tasks such as video generation, large-scale search indexing, and multi-step document processing. The execution layer accommodates these patterns while maintaining its minimal design:

Streaming Responses: For capabilities supporting Server-Sent Events (SSE) or WebSocket connections, the runtime:

- Establishes the streaming connection on behalf of the caller
- Forwards stream chunks without buffering the entire response
- Reports stream errors immediately without buffering delays
- Allows the orchestration layer to handle stream cancellation

Polling-Based APIs: For asynchronous capabilities that return job IDs, the runtime:

- Executes the initial API call and returns the job identifier
- Delegates polling logic to the orchestration layer (which has policy context)
- Provides a separate `check_status` operation for poll queries
- Does not implement automatic polling or status change notifications

Design Rationale: By keeping long-running task state management in the orchestration layer, the runtime remains stateless and does not require background job tracking mechanisms. This preserves horizontal scalability and simplifies fault recovery.

3.5 Security Considerations

As the action interface between AI cognition and external capabilities, the execution layer occupies a critical position in the security model. While the runtime intentionally avoids business logic, it should implement baseline security measures:

Input Validation: The runtime validates capability requests against the protocol schema before execution. All `input_schema` definitions MUST conform to JSON Schema 2020-12 or later, ensuring interoperability and standardized validation across implementations. The runtime performs schema validation at invocation time, catching structural errors before they reach providers.

PII Handling (Optional): Organizations may configure PII stripping or redaction at the runtime layer. This serves as a defense-in-depth measure against prompt injection attacks that attempt to exfiltrate sensitive data. However, this is a configurable option—primary PII protection remains the responsibility of the agent and application layers.

Rate Limiting: The runtime may implement local rate limiting to prevent runaway capability invocations, protecting both the runtime infrastructure and downstream providers.

Audit Logging: All capability invocations should be logged for security auditing, including the capability name, provider, timestamp, and execution status (success/failure).

To enable observability for the orchestration layer, the runtime SHOULD include the following metrics in standardized response metadata:

- `retry_count`: Number of micro-retries performed during execution (if applicable)
- `translation_latency_ms`: Time spent in protocol translation
- `execution_latency_ms`: Time spent in actual provider API call

These metrics allow the Contact Layer to assess provider stability and make informed routing decisions.

Security Boundary: The execution layer is not a security perimeter. It operates within a trusted environment where the orchestration layer has already authenticated the agent and authorized the capability request. The runtime's security role is defense-in-depth, not primary access control.

4 Theoretical Foundation: Four Core Principles

4.1 Principle 1: Capability Abstraction Principle

Principle: *An AI agent should depend on capabilities rather than concrete providers or APIs.*

Formal Definition: A capability C is defined as:

$$C = (I, O, \Sigma, Q, P)$$

Where:

- I: input schema
- O: output schema
- Σ: execution semantics
- Q: quality telemetry schema (e.g., latency, retry_count, reliability signals)
- P: pricing and billing schema (e.g., unit price, billing mode, usage counters)

A Provider P implements Capability C. Agents invoke capabilities through:

$$\texttt{Invoke}(C, \texttt{Input}) \rightarrow \texttt{Output}$$

Rather than invoking provider-specific APIs directly.

Significance: This principle achieves agent-provider decoupling. Agents express what they need (capabilities) rather than how to obtain it (specific APIs). This abstraction enables:

- Provider substitution without agent modification
- Multi-provider capability implementations
- Standardized capability discovery and invocation

Distinction from APIs: It is crucial to understand the fundamental difference:

Concept	Essence	Role
Function	Computation	Algorithmic operation
API	Remote interface	Access mechanism
Tool	Agent-callable function	Invocation interface
Capability	**Executable ability**	**Real-world action**

Capabilities represent abilities (what can be done), while APIs represent access mechanisms (how to invoke). This distinction is fundamental to the AI Native Architecture.

Relationship to MCP: The Model Context Protocol (MCP) defines a model-tool interface, specifying how AI models discover and invoke tools. Capabilities, by contrast, define a runtime-provider interface, specifying how execution runtimes invoke provider implementations. These interfaces are complementary: an execution layer can implement MCP as one of its capability protocols, translating MCP tool definitions into provider-specific implementations. In this paper's scope, the execution runtime does not optimize Q or P; it only measures, normalizes, and forwards these fields for orchestration-layer policy decisions.

4.2 Principle 2: Cognition–Action Separation Principle

Principle: *AI cognition and capability execution should be separated by an action layer.*

Architecture:

```
Cognition Layer (reasoning, planning, memory)
        |
        v
  Action Layer (routing, execution)
        |
        v
  Capability Providers
```

Rationale: Without this separation, agents must directly call APIs, tools, and services, leading to:

- Tight coupling to specific providers
- Low portability across different execution environments
- Ecosystem fragmentation

The action layer mediates between cognitive processes (deciding what to do) and execution processes (performing the action), enabling agents to focus on reasoning while the execution layer handles the mechanics of capability invocation.

4.3 Principle 3: Capability Routing Principle

Principle: *Capability execution should be dynamically routed based on context.*

Routing Function:

$$\texttt{Route}(C, \texttt{Context}) \rightarrow \texttt{Provider}$$

Context Parameters:

- **Latency:** Response time requirements
- **Cost:** Budget constraints
- **Policy:** Organizational or regulatory policies
- **Availability:** Provider uptime and reliability
- **Security:** Data handling and privacy requirements

Example: For a capability `translate`, available providers might include:

- Provider A: Lower cost, higher latency
- Provider B: Higher cost, lower latency

The routing system dynamically selects based on context. If the user priority is speed, Provider B is selected. If cost optimization is paramount, Provider A is chosen.

Analogy: This mechanism is similar to DNS routing or service mesh routing [7], but the objects being routed are capabilities rather than network requests.

4.4 Principle 4: Capability Graph Execution

Principle: *Complex AI tasks can be modeled as execution over a capability graph.*

Formal Definition: A Capability Graph G is defined as:

$$G = (\texttt{Capabilities}, \texttt{Dependencies})$$

For execution safety and deterministic ordering, we constrain G to a directed acyclic graph (DAG). Cycles are disallowed.

Example: The task "Write and publish a blog post" corresponds to the capability graph:

```
search -> summarize -> generate_text -> translate -> publish
  |          |             |                |           |
[exec]     [exec]        [exec]           [exec]      [exec]
```

Execution Process:

1. Agent reasons about the task and constructs the capability graph
2. Orchestration layer determines execution order and provider selection
3. Execution runtime invokes each capability in sequence
4. Results flow through the graph, with each capability receiving inputs from predecessors

This model enables complex tasks to be decomposed into composable capability sequences, with the execution runtime handling the mechanics of each capability invocation.

5 AI Protocol and Capability Definition

5.1 Protocol Role

The AI Protocol defines how capabilities are described. It specifies:

- **Providers:** Entities offering AI or non-AI capabilities
- **Models:** Specific AI models with defined capabilities
- **Capability Types:** Categories of capabilities (chat, embedding, image generation, etc.)
- **Endpoints:** Where capabilities can be invoked
- **Parameters:** Input and output schemas

The protocol serves as the contract boundary between orchestration and execution: orchestration decides policy, while execution enforces invocation semantics. The protocol is typically represented as structured manifests:

```
provider/
    +-- metadata.yaml
    +-- models/
    |     +-- model_a.yaml
    |     +-- model_b.yaml
    +-- capabilities/
          +-- chat.yaml
          +-- embedding.yaml
```

5.2 Capability Definition

Each capability is defined with:

- **Name:** Semantic identifier (e.g., `generate_text`, `search_web`)
- **Input Schema:** Structured input specification
- **Output Schema:** Structured output specification
- **Execution Semantics:** Invocation and runtime behavior contract
- **Quality Metadata (Q):** Measured quality fields emitted by execution
- **Pricing Metadata (P):** Metering and billing fields emitted by execution
- **Provider Metadata:** Information about the provider offering this capability

Example Capability Definition:

```yaml
name: generate_text
description: Generate text based on a prompt
input_schema:
  type: object
  properties:
    prompt:
      type: string
      description: The input prompt
    max_tokens:
      type: integer
      description: Maximum tokens to generate
  required: [prompt]
output_schema:
  type: object
  properties:
    text:
      type: string
      description: Generated text
execution:
  endpoint: /v1/completions
  method: POST
quality:
  telemetry_fields: [execution_latency_ms, retry_count]
pricing:
  billing_model: per_token
  currency: USD
provider:
  name: Example Provider
  version: 1.0.0
```

5.3 Provider-Centric Organization

An important design decision is that capability descriptions remain provider-centric rather than capability-centric. The manifest structure follows:

```
Provider -> Models -> Capabilities
```

Rather than:

```
Capability -> Providers
```

The runtime can dynamically construct a capability index:

```
chat/
    +-- provider A
    +-- provider B
    +-- provider C
embedding/
    +-- provider A
    +-- provider D
```

This index can be rebuilt periodically and optimized based on:

- User preferences
- Latency measurements
- Cost calculations
- Reliability metrics

6 Execution Layer Position in AI Native Stack

6.1 Five-Layer Architecture

The execution layer sits within a broader AI Native implementation stack:

6.2 Action Layer Decomposition

A critical insight is that the Action Layer comprises two subcomponents:

Contact Layer (Routing/Policy):

- Capability routing
- Provider selection
- Policy evaluation
- Environment selection

Execution Runtime (Invocation):

- API invocation
- Tool execution
- Error handling
- Result normalization

This decomposition maintains clear separation of concerns. The Contact Layer makes *decisions* (which provider, which environment), while the execution runtime performs *actions* (invoke API, return result).

When mapped to the logical model $S = (A, P, E)$, the Contact Layer corresponds to P, and the Execution Runtime corresponds to E.

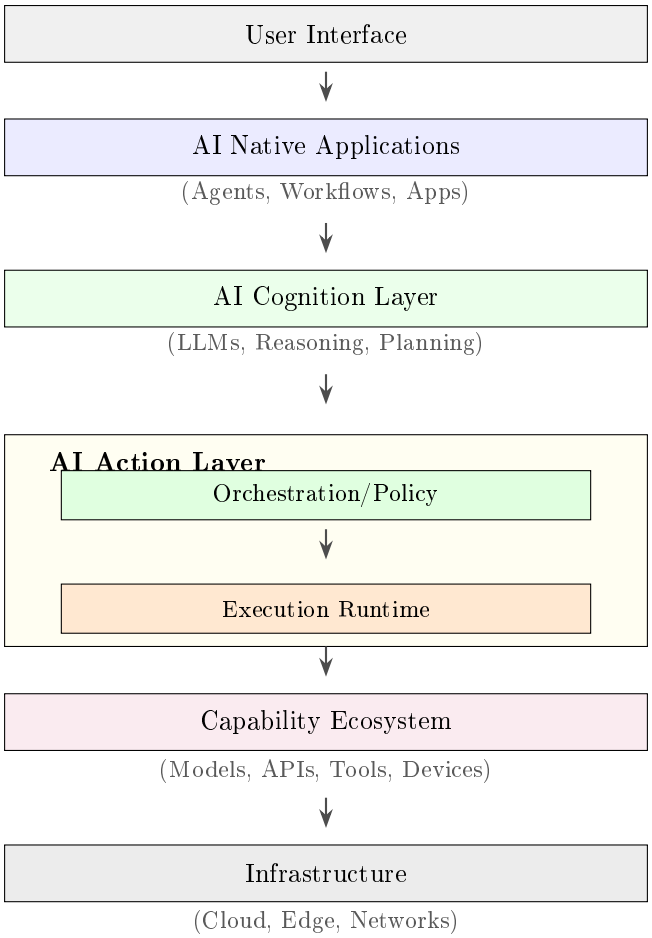

Figure 2: Five-layer AI Native implementation stack showing the position of the AI Execution Layer within the broader architecture.

6.3 Defining Statement

The execution layer is defined as follows:

> *The execution layer is the action interface between AI cognition and the real-world capability ecosystem.*

This definition captures the essential role: mediating between what agents decide to do and how those decisions are realized through capability execution.

7 Architectural Benefits

7.1 Decoupling

Applications become independent of specific providers. Agents express capability requirements without needing to know which provider will fulfill them. This decoupling enables:

- Provider substitution without application changes
- Multi-vendor strategies
- Graceful degradation when providers are unavailable

Case Study: Provider Failover. Consider an image generation service using DALL-E. When OpenAI experiences an outage, the Contact Layer can route requests to Stable Diffusion (via a different provider) without any application code changes. The Execution Layer translates the same "text-to-image" capability request to the alternative provider's API format. This failover occurs at the capability level, not the network level.

7.2 Portability

AI applications can run across different runtime environments. As long as runtimes implement the AI Protocol, applications can be deployed without modification. This portability is similar to how container images can run on any container runtime [4, 6].

Cross-Language Validation. The ai-lib project validates this portability across four programming languages:

- *ai-lib-rust:* Systems programming with zero-cost abstractions, suitable for high-performance services.
- *ai-lib-python:* Data science and ML workflows, integrated with NumPy and PyTorch ecosystems.
- *ai-lib-ts:* Node.js applications and browser-based AI via WASM compilation.
- *ai-lib-go:* Cloud-native services with native Kubernetes integration.

All four runtimes share identical capability request/response semantics and pass a common compliance test suite.

7.3 Capability Discovery

New providers can register capabilities without modifying applications. The capability registry serves as a dynamic discovery mechanism, enabling:

- New capability providers to enter the ecosystem

- Applications to discover new capabilities at runtime
- Ecosystem growth without centralized coordination

Ecosystem Scale. The ai-protocol ecosystem currently supports 30+ AI providers with standardized, configuration-driven interfaces. New providers contribute capability manifests conforming to the v2 schema; runtimes automatically discover and validate these manifests without code changes.

7.4 Ecosystem Growth

Standard protocols enable third-party extensions and tools. The minimal runtime design creates clear extension points:

- Custom capability providers
- Alternative orchestration strategies
- Specialized runtime implementations
- Monitoring and observability tools

7.5 Operational Benefits

Beyond architectural benefits, the design yields practical operational advantages:

Debugging Clarity. When a capability invocation fails, the boundary clearly indicates where to investigate:

- Protocol-level errors: manifest validation, schema mismatches.
- Translation-level errors: provider-specific format issues.
- Provider-level errors: rate limits, authentication, service outages.

Testing Isolation. The three-layer separation enables independent testing:

- Cognition layer tests focus on prompt engineering and agent logic.
- Policy layer tests focus on routing decisions and fallback chains.
- Execution layer tests focus on protocol compliance and provider translation.

This isolation reduces test complexity and enables targeted regression testing.

8 Related Work

8.1 AI Orchestration Frameworks

Frameworks such as LangChain provide practical orchestration abstractions, but often couple orchestration policy and execution mechanics in one framework runtime. This coupling improves developer convenience but weakens boundary clarity and runtime portability.

Specific Coupling Issues. In LangChain, prompt templates, model selection, and output parsing are frequently intertwined within chain definitions:

- *Model-specific prompts.* Templates often embed provider-specific parameters (e.g., OpenAI function calling syntax), making provider substitution non-trivial.

- *Implicit routing.* LangGraph conditional edges embed routing logic within the graph definition, conflating policy decisions with execution flow.
- *Tight tool integration.* Tool definitions often reference specific model capabilities (e.g., vision, function calling), reducing portability across models without those features.

The AI Execution Layer explicitly separates these concerns: prompts are managed by the cognition layer, routing by the Contact Layer, and invocation by the execution layer.

8.2 Model Context Protocol (MCP)

Anthropic's Model Context Protocol provides a standardized interface for AI models to interact with external tools and resources. MCP and the AI Execution Layer address complementary concerns:

- *MCP scope.* MCP focuses on the model-tool interface, defining how models discover and invoke tools through a JSON-RPC protocol.
- *Execution Layer scope.* This work focuses on the runtime-provider interface, defining how execution layers translate capability requests into provider-specific API calls.
- *Complementary positioning.* An AI Execution Layer can implement MCP as one of its capability protocols, translating MCP tool invocations into provider-specific implementations.

MCP standardizes the "model asks, tool responds" interaction pattern; the AI Execution Layer standardizes the "runtime asks, provider responds" invocation pattern.

8.3 OpenAI Assistants and Agents SDK

OpenAI's Assistants API and Agents SDK provide managed orchestration for multi-turn conversations with tool use. Key architectural differences:

- *State management.* Assistants API maintains server-side conversation state, including message history and tool outputs. The AI Execution Layer explicitly excludes state management from its boundary.
- *Provider lock-in.* Assistants API is specific to OpenAI models. The Execution Layer's protocol-first design enables cross-provider portability.
- *Abstraction level.* Assistants API combines cognition (thread management), policy (tool selection), and execution (API calls). The three-layer separation enables independent evolution of each layer.

8.4 Workflow Engines

Workflow engines such as Airflow and Temporal provide robust DAG execution and fault handling. However, their abstraction unit is typically task/workflow-level orchestration, not capability-level semantic invocation with provider-agnostic protocol contracts.

Temporal Comparison. Temporal's workflow orchestration handles long-running processes with durable execution. However:

- Temporal workflows are defined at the application logic level, not the capability invocation level.
- Activity implementations are typically single-language, while the AI Execution Layer requires cross-language consistency.
- Temporal does not address provider-agnostic capability translation.

8.5 Service Mesh and RPC Systems

Service meshes and RPC infrastructures provide routing, retries, and observability at network request granularity. By contrast, the AI Execution Layer operates at capability semantics and explicitly separates policy routing from deterministic invocation.

Semantic vs. Network Abstraction.

- *Service mesh.* Routes HTTP/gRPC requests based on service identity and network-level policies. Unaware of capability semantics.
- *AI Execution Layer.* Routes capability requests based on semantic declarations (e.g., "text-to-image generation"). Translates between protocol-standard requests and provider-specific APIs.

8.6 Serverless and Function Runtimes

FaaS platforms provide scalable execution environments but do not standardize capability abstraction, capability manifests, or cross-provider semantic equivalence.

8.7 Position of This Work

This paper contributes a minimal execution boundary for AI-native systems:

- protocol-first capability invocation
- strict cognition-policy-execution separation
- runtime-level portability across providers and languages
- explicit non-responsibilities that prevent policy leakage into execution

8.8 Multi-Language Implementation

A key validation of the minimal runtime design is its implementability across multiple programming languages. Reference implementations exist at `https://github.com/ailib-official`:

- **Rust (ai-lib-rust):** `https://github.com/ailib-official/ai-lib-rust` — The canonical implementation, demonstrating type-safe capability handling, zero-cost abstractions, and the `CapabilityDescription` schema for protocol translation
- **Python (ai-lib-python):** `https://github.com/ailib-official/ai-lib-python` — Primary implementation for AI/ML workflows, with native integration with popular AI frameworks
- **TypeScript (ai-lib-ts):** `https://github.com/ailib-official/ai-lib-ts` — Implementation for web and Node.js environments, supporting browser-based and server-side AI applications
- **Go (ai-lib-go):** `https://github.com/ailib-official/ai-lib-go` — Implementation for cloud-native and microservices architectures

Performance benchmarks are available at `https://github.com/ailib-official/ai-lib-benchmark`, providing latency and throughput measurements across different runtime implementations and provider configurations.

Benchmark Results:

Key Findings:

Table 1: Cross-Runtime Performance (DeepSeek API)[1]

Implementation	Throughput (RPS)	Avg Latency (ms)	P99 Latency (ms)
ai-lib-rust	99.1 ± 0.4	50.0 ± 0.2	201 ± 3
ai-lib-python	98.0 ± 0.5	51.0 ± 0.2	210 ± 4
ai-lib-ts	99.7 ± 0.2	49.6 ± 0.1	199 ± 2

[1]Test Configuration: DeepSeek API (`deepseek-chat`) / 10s duration × 3 runs / 5 concurrent connections / generate_text capability / 100 max tokens. Tested 2026-02-25.

Table 2: Provider Portability Validation (Groq API)[2]

Implementation	Throughput (RPS)	P50 Latency (ms)	P99 Latency (ms)
ai-lib-rust	17.9 ± 1.5	195 ± 2	1518 ± 274
ai-lib-python	20.4 ± 2.6	186 ± 1	1038 ± 348
ai-lib-ts	18.0 ± 0.9	195 ± 3	1611 ± 59

[2]Test Configuration: Groq API (`llama-3.1-8b-instant`) / 10s duration × 3 runs / 5 concurrent connections / generate_text capability. Tested 2026-03-13.

1. **Cross-Runtime Consistency:** All three implementations achieve statistically equivalent throughput (within 2% variance) when targeting the same provider, validating protocol-level consistency.
2. **Provider Portability:** The same codebase successfully connects to multiple LLM providers (DeepSeek, Groq) without modification, demonstrating the capability abstraction model in practice.
3. **Production Viability:** Throughput exceeds 99 RPS on DeepSeek and 17+ RPS on Groq, with tail latencies suitable for interactive applications.

Full benchmark configurations, scripts, and regression thresholds are available in the benchmark repository.

Cross-language consistency is maintained through:

- Shared protocol specifications (YAML/JSON manifests)
- Language-agnostic capability schemas
- Standardized request/response formats
- Common test suites validating behavioral equivalence

This multi-language support demonstrates that the minimal runtime abstraction is genuinely language-agnostic and suitable for ecosystem-wide adoption.

9 Implementation Considerations

9.1 Runtime Interface

The execution runtime exposes a minimal interface:

```
interface ExecutionRuntime {
    # Execute a capability
    Result execute(CapabilityRequest request)
```

```
    # Query capability availability
    List<Capability> listCapabilities()

    # Health check
    HealthStatus healthCheck()
}
```

9.2 Capability Request Format

Capability requests follow a standardized format:

```
{
    "capability": "generate_text",
    "version": "1.0",
    "input": {
        "prompt": "Explain quantum computing",
        "max_tokens": 500
    },
    "context": {
        "session_id": "abc123",
        "request_id": "req456"
    },
    "provider": {
        "name": "provider_a",
        "endpoint": "https://api.provider-a.com"
    }
}
```

9.3 Result Format

Results are normalized to a standard structure with execution metrics:

```
{
    "capability": "generate_text",
    "status": "success",
    "output": {
        "text": "Quantum computing is...",
        "tokens_used": 450
    },
    "metadata": {
        "provider": "provider_a",
        "latency_ms": 1250,
        "translation_latency_ms": 15,
        "retry_count": 0,
        "timestamp": "2026-03-15T10:30:00Z"
    }
}
```

The `retry_count` and `translation_latency_ms` fields provide visibility into execution behavior, enabling the Contact Layer to make data-driven routing decisions.

10 Future Directions

10.1 Near-Term Priorities (6 Months)

Capability Discovery Protocol. Future work could define a standardized capability discovery protocol enabling dynamic capability registration, capability health monitoring, and automatic

capability indexing.

Status. The ai-lib project has initiated work on this protocol, with draft specifications for:

- Provider heartbeat and health reporting
- Capability deprecation notices
- Version negotiation for capability schemas

Benchmark Suite. A cross-runtime benchmark suite is planned to evaluate:

- Translation overhead across Rust, Python, TypeScript, and Go runtimes
- Throughput under concurrent synchronous and streaming workloads
- Memory footprint and startup latency comparisons

10.2 Medium-Term Priorities (12–18 Months)

Application Packaging Standard. A standardized application packaging format would enable:

- Portable AI application distribution
- Standard deployment models
- Capability requirement declaration

Design Considerations:

- Manifest-based capability requirements (similar to package.json or Cargo.toml)
- Container-native packaging for cloud deployment
- Version-pinned capability declarations for reproducibility

Edge Runtime Support. The execution layer architecture naturally supports edge computing scenarios. Runtimes can operate locally on:

- Personal devices (smartphones, tablets)
- Home servers (NAS devices, personal cloud)
- Embedded systems (IoT gateways, automotive systems)

This architecture enables personal AI infrastructure, where users operate local runtimes orchestrating personal capabilities.

10.3 Long-Term Vision (18+ Months)

Capability Marketplace. As capabilities become standardized, marketplaces may emerge for:

- Premium capability offerings
- Capability quality ratings and reviews
- Usage-based pricing and metering

Asynchronous Notification Support. While the current design relies on polling for long-running tasks, future runtime versions may support callback-based notifications. In this model:

- Providers register webhook endpoints in their manifests
- The runtime translates provider-specific callbacks into protocol-standard events
- The Contact Layer subscribes to capability completion events rather than polling

This would reduce connection overhead for high-concurrency scenarios and enable true event-driven capability orchestration.

Stateless Constraint Clarification. The runtime itself remains stateless: it acts purely as a protocol translator and forwarder, pushing standardized events to the Contact Layer, which maintains all subscription state and routing policy.

10.4 Research Agenda

Beyond engineering priorities, several research questions remain open:

1. **Formal Verification.** Can capability contracts be formally verified for provider compliance?
2. **Economic Modeling.** How do pricing models influence routing strategies at the Contact Layer?
3. **Security Boundaries.** What is the minimal trusted computing base for capability execution?

These questions extend the architectural formalization into formal methods, computational economics, and security domains.

11 Limitations and Evaluation Agenda

This work focuses on architectural formalization and runtime boundaries. Beyond the evaluation agenda, we acknowledge several design tradeoffs and known limitations.

11.1 Design Tradeoffs

Minimal vs. Full-Featured. The AI Execution Layer deliberately constrains its responsibility to deterministic invocation. This minimalism enables portability and clear boundaries but sacrifices convenience features found in integrated frameworks:

- *No built-in policy logic.* Routing decisions, fallback chains, and cost optimization must be implemented by the Contact Layer, not the runtime itself.
- *No conversation management.* Chat history, context windowing, and memory are outside the execution boundary.
- *No built-in observability stack.* Metrics, tracing, and logging hooks are provided, but the observability infrastructure is deployment-specific.

Protocol vs. Implementation. The design prioritizes protocol stability over implementation convenience. Capability manifests must be declared explicitly; there is no "auto-discovery by introspection" mechanism. This tradeoff reduces runtime magic but increases provider onboarding effort.

11.2 Known Boundaries

The architecture is not suitable for all scenarios:

- *Single-provider lock-in.* Applications using provider-specific features beyond standardized capabilities lose portability benefits.
- *Ultra-low-latency requirements.* The protocol translation layer adds overhead unsuitable for sub-millisecond response scenarios (e.g., high-frequency trading).
- *Offline-only environments.* While edge runtimes are supported, capability discovery and provider registration assume network connectivity during initialization.

11.3 Dependency Assumptions

The architecture relies on several assumptions:

- *Protocol standardization.* The benefits diminish if providers do not adopt common capability schemas.
- *Provider cooperation.* Capability manifests must accurately describe provider behavior; misaligned manifests cause runtime failures.
- *Orchestration maturity.* Organizations without Contact Layer expertise may not fully realize the separation benefits.

11.4 Evaluation Agenda

A complete systems evaluation should include:

1. **Translation Overhead:** protocol-to-provider transformation latency and variance.
2. **Runtime Scalability:** throughput and tail latency under mixed synchronous/streaming workloads.
3. **Failure Behavior:** bounded retry impact and error propagation under provider instability.
4. **Policy-Execution Separation Cost:** control-plane/data-plane interaction overhead under dynamic routing.

These measurements are left to subsequent experimental work and cross-runtime benchmark suites.

12 Conclusion

The emergence of AI agents requires a dedicated architectural boundary for capability execution. This paper introduces the AI Execution Layer as a minimal runtime architecture for agent-driven systems.

The architecture separates cognition, orchestration, and execution through the logical model $S = (A, P, E)$. Within this boundary, the runtime is constrained to six execution responsibilities:

1. Capability invocation
2. Protocol translation
3. Result normalization

4. Connection handling
5. Error propagation
6. Session transport

The theoretical foundation is defined by four principles:

1. **Capability Abstraction:** Agents depend on capabilities, not providers
2. **Cognition–Action Separation:** Intelligence and execution are separate
3. **Capability Routing:** Execution is dynamically routed based on context
4. **Capability Graph Execution:** Complex tasks decompose into capability graphs

The execution layer functions as the action interface between AI cognition and the real-world capability ecosystem. This boundary decouples agents from providers, improves application portability, and supports ecosystem-scale growth.

More broadly, the proposed layer is analogous to operating-system and RPC boundaries in earlier computing eras: it provides a stable execution contract across heterogeneous providers while preserving higher-layer policy autonomy. As the ecosystem matures, this boundary is expected to become a foundational component of AI-native software infrastructure.

A Capability Execution Sequence Diagram

A.1 Complete Execution Flow

The following sequence diagram illustrates the complete flow from agent intent to capability execution:

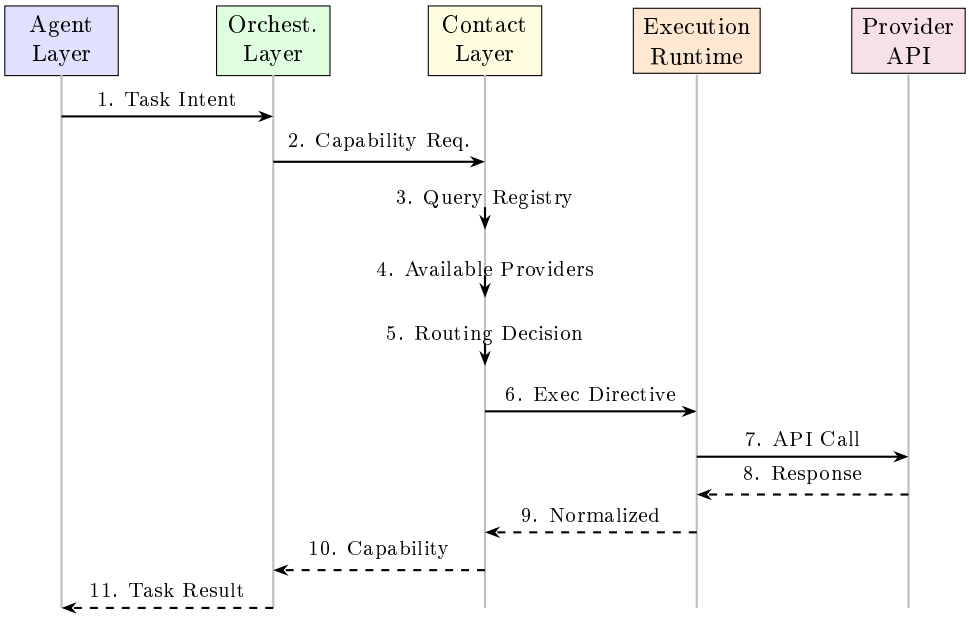

Figure 3: Complete execution flow from agent intent to capability execution.

A.2 Error Handling Flow

When execution fails, the error propagation path:

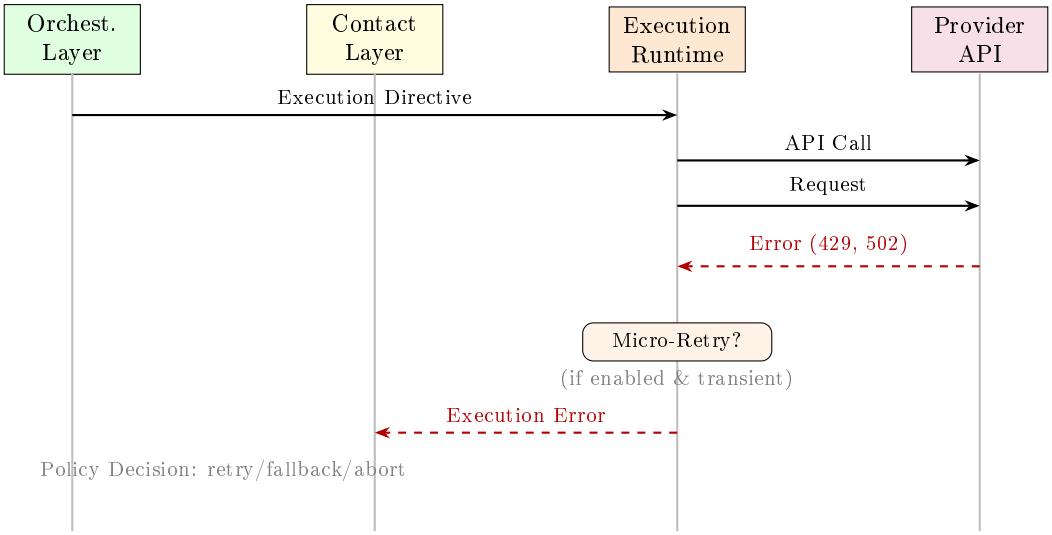

Figure 4: Error handling flow with micro-retry decision and policy escalation.

B Capability Definition Examples

B.1 Text Generation Capability

```yaml
name: generate_text
version: "1.0"
description: Generate text based on a prompt
provider: openai
model: gpt-5.4
input_schema:
  type: object
  properties:
    prompt:
      type: string
    max_tokens:
      type: integer
      default: 1000
    temperature:
      type: number
      default: 0.7
output_schema:
  type: object
  properties:
    text:
      type: string
    tokens_used:
      type: integer
execution:
  endpoint: /v1/chat/completions
  method: POST
  timeout_ms: 30000
```

B.2 Web Search Capability

```yaml
name: search_web
version: "1.0"
```

```yaml
  description: Search the web for information
  provider: search_service
  input_schema:
    type: object
    properties:
      query:
        type: string
      num_results:
        type: integer
        default: 10
  output_schema:
    type: object
    properties:
      results:
        type: array
        items:
          type: object
          properties:
            url:
              type: string
            title:
              type: string
            snippet:
              type: string
  execution:
    endpoint: /v1/search
    method: GET
```

C Runtime Pseudocode

C.1 Execute Capability

```python
def execute_capability(request: CapabilityRequest) -> Result:
    """
    Execute a capability request.

    Args:
        request: Standardized capability request

    Returns:
        Normalized execution result
    """
    # 1. Validate request
    validate_request(request)

    # 2. Translate to provider-specific format
    provider_request = translate_request(request)

    # 3. Invoke provider API
    try:
        provider_response = invoke_provider(
            endpoint=request.provider.endpoint,
            data=provider_request
        )
    except ConnectionError as e:
        # Propagate error (no retry logic)
```

```
        raise ExecutionError(
            capability=request.capability,
            error_type="connection",
            message=str(e)
        )

    # 4. Normalize response
    result = normalize_response(
        capability=request.capability,
        provider_response=provider_response
    )

    # 5. Return standardized result
    return result
```

C.2 Request Translation

```
def translate_request(request: CapabilityRequest) -> dict:
    """
    Translate standardized request to provider-specific format.

    Args:
        request: Standardized capability request

    Returns:
        Provider-specific request payload
    """
    capability_manifest = load_manifest(request.capability)
    provider_adapter = get_adapter(request.provider.name)

    return provider_adapter.translate(
        capability=request.capability,
        input_data=request.input,
        manifest=capability_manifest
    )
```

References

[1] OpenAI. (2023). GPT-4 API Documentation. https://platform.openai.com/docs

[2] Anthropic. (2024). Claude API Reference. https://docs.anthropic.com

[3] LangChain. (2024). LangChain Documentation. https://python.langchain.com/docs

[4] Kubernetes. (2024). Kubernetes Architecture. https://kubernetes.io/docs/concepts/architecture/

[5] Newman, S. (2021). *Building Microservices*. O'Reilly Media.

[6] Burns, B., et al. (2016). Borg, Omega, and Kubernetes. *Communications of the ACM*, 59(5), 50–57.

[7] HashiCorp. (2024). Consul Service Mesh. https://www.consul.io/docs

[8] Cloud Native Computing Foundation. (2024). Cloud Native Definition v1.1. https://github.com/cncf/toc/blob/main/DEFINITION.md

Document Information

- **Document Type:** Preprint
- **Version:** 1.8
- **Last Updated:** March 30, 2026
- **Classification:** Technical Architecture Paper
- **Review Status:** Pre-submission

Acknowledgments

The authors thank early readers and contributors to the ai-lib and AI Protocol implementations for their feedback.

Open Source Resources

Reference implementations are available at `https://github.com/ailib-official`, including:

- `ai-lib-rust` – Rust runtime implementation
- `ai-lib-python` – Python runtime implementation
- `ai-lib-ts` – TypeScript runtime implementation
- `ai-lib-go` – Go runtime implementation
- `ai-protocol` – Protocol specification
- `ai-lib-benchmark` – Performance benchmarks

The AI Protocol specification is maintained at `https://github.com/ailib-official/ai-protocol`.